

Solving problems with polymorphism

Polymorphism means different objects can respond to the same “message” in different ways — that is, the objects share a method, but each has its own behaviour when that method is called. This can be achieved in Java when multiple classes implement the same interface.

This sort of polymorphism is not possible in the imperative programming paradigm. When you call a procedure, you know exactly which procedure (and hence, which instructions) will be executed.

To see why polymorphism is useful for designing software and solving problems, let’s consider a simple model of the four directions on a compass: North, East, South and West. Each direction will have the following methods:

- getName returns the name of the direction, as a string.
- rotateClockwise returns a direction object representing the direction 90 degrees clockwise from this one.
- rotateCounterClockwise returns a direction object representing the direction 90 degrees counter-clockwise from this one.

An imperative model

To begin with, we'll write the model as just one class named `Direction`, and the behaviour will be implemented using imperative code — that is, lots of `if/else` blocks:

```
1: public class Direction {
2:     private final String name;
3:
4:     private Direction(String name) {
5:         // regex to check all four options at once
6:         if(!name.matches("North|East|South|West")) {
7:             String message = "Unknown direction: " + name;
8:             throw new IllegalArgumentException(message);
9:         }
10:        this.name = name;
11:    }
12:
13:    public String getName() {
14:        return name;
15:    }
16:
17:    public Direction rotateClockwise() {
18:        if("North".equals(name)) {
19:            return new Direction("East");
20:        } else if("East".equals(name)) {
21:            return new Direction("South");
22:        } else if("South".equals(name)) {
23:            return new Direction("West");
24:        } else {
25:            return new Direction("North");
26:        }
27:    }
28:
29:    public Direction rotateCounterClockwise() {
30:        if("North".equals(name)) {
31:            return new Direction("West");
32:        } else if("East".equals(name)) {
33:            return new Direction("North");
34:        } else if("South".equals(name)) {
35:            return new Direction("East");
36:        } else {
37:            return new Direction("South");
38:        }
39:    }
40: }
```

This `Direction` class works fine, but it is a bit complicated. (Imagine a similar scenario where there are forty directions instead of four!) Really, each of the four `Direction` objects does different things when you call its methods, we have written four different behaviours in each method.

A polymorphic model

Different Direction objects should have different behaviour, so we will refactor Direction as an interface:

```
1: public interface Direction {
2:     public String getName();
3:
4:     public Direction rotateClockwise();
5:
6:     public Direction rotateCounterClockwise();
7: }
```

The interface has the same methods as the class used to have, but the implementations belong in separate classes for each direction. Here is the North class — the East, South and West classes will be very similar:

```
1: public class North implements Direction {
2:     @Override
3:     public String getName() {
4:         return "North";
5:     }
6:
7:     @Override
8:     public Direction rotateClockwise() {
9:         return new East();
10:    }
11:
12:    @Override
13:    public Direction rotateCounterClockwise() {
14:        return new West();
15:    }
16: }
```

This is a much simpler design — there is no need for if/else blocks to decide how to rotate clockwise or counter-clockwise, because each class only needs to return the correct result for itself.